配置系统、Feature 模块与 MCP 体系

配置系统

多级配置合并

```
优先级（高 → 低）：
  Project   (.opencode/oh-my-opencode.jsonc)
    → User  (~/.config/opencode/oh-my-opencode.jsonc)
      → Defaults（内置默认值）
```

格式：JSONC（支持注释），Zod v4 验证，snake_case 键名。

配置加载流程

> 源文件：`src/plugin-config.ts`（180 行）— `loadPluginConfig()` 完整实现

```ts
// src/plugin-config.ts — 配置加载入口
export function loadPluginConfig(
  directory: string,
  ctx: unknown
): OhMyOpenCodeConfig {
  // User-level config path — prefer .jsonc over .json
  const configDir = getOpenCodeConfigDir({ binary: "opencode" })
  const userBasePath = path.join(configDir, "oh-my-opencode")
  const userDetected = detectConfigFile(userBasePath)       // 自动探测 .jsonc/.json
  const userConfigPath =
    userDetected.format !== "none"
      ? userDetected.path
      : userBasePath + ".json"

  // Project-level config path — prefer .jsonc over .json
  const projectBasePath = path.join(directory, ".opencode", "oh-my-opencode")
  const projectDetected = detectConfigFile(projectBasePath)
  const projectConfigPath =
    projectDetected.format !== "none"
      ? projectDetected.path
      : projectBasePath + ".json"

  // Load user config first (base)
  let config: OhMyOpenCodeConfig =
    loadConfigFromPath(userConfigPath, ctx) ?? {}

  // Override with project config
  const projectConfig = loadConfigFromPath(projectConfigPath, ctx)
  if (projectConfig) {
    config = mergeConfigs(config, projectConfig)  // project 覆盖 user
  }

  return config
}
```

配置合并策略 — `mergeConfigs()` 实现：

```
// src/plugin-config.ts — 配置合并逻辑
export function mergeConfigs(
  base: OhMyOpenCodeConfig,
  override: OhMyOpenCodeConfig
): OhMyOpenCodeConfig {
  return {
    ...base,
    ...override,
    agents: deepMerge(base.agents, override.agents),       // 深度合并
    categories: deepMerge(base.categories, override.categories), // 深度合并
    disabled_agents: [                                      // 去重合并
      ...new Set([...(base.disabled_agents ?? []), ...(override.disabled_agents ?? [])]
    ],
    disabled_mcps: [                                        // 去重合并
      ...new Set([...(base.disabled_mcps ?? []), ...(override.disabled_mcps ?? [])]),
    ],
    disabled_hooks: [ ...new Set([...(base.disabled_hooks ?? []), ...(override.disabled
    disabled_commands: [ ...new Set([...(base.disabled_commands ?? []), ...(override.di
    disabled_skills: [ ...new Set([...(base.disabled_skills ?? []), ...(override.disabl
    claude_code: deepMerge(base.claude_code, override.claude_code),
  }
}
```

容错加载 — `parseConfigPartially()` 实现按字段部分解析：

```ts
// src/plugin-config.ts — 部分配置加载（无效字段跳过，不会导致全部失败）
export function parseConfigPartially(
  rawConfig: Record<string, unknown>
): OhMyOpenCodeConfig | null {
  const fullResult = OhMyOpenCodeConfigSchema.safeParse(rawConfig)
  if (fullResult.success) return fullResult.data

  const partialConfig: Record<string, unknown> = {}
  const invalidSections: string[] = []
  for (const key of Object.keys(rawConfig)) {
    const sectionResult = OhMyOpenCodeConfigSchema.safeParse({ [key]: rawConfig[key] })
    if (sectionResult.success) {
      const parsed = sectionResult.data as Record<string, unknown>
      if (parsed[key] !== undefined) partialConfig[key] = parsed[key]
    } else {
      invalidSections.push(`${key}: ${sectionResult.error.issues.filter(i => i.path[0]
    }
  }
  return partialConfig as OhMyOpenCodeConfig
}
```

根 Schema 结构

> 源文件：`src/config/schema/oh-my-opencode-config.ts`（68 行，完整源码）

```typescript
// src/config/schema/oh-my-opencode-config.ts — 完整 Schema 定义
export const OhMyOpenCodeConfigSchema = z.object({
  $schema: z.string().optional(),
  /** Enable new task system (default: false) */
  new_task_system_enabled: z.boolean().optional(),
  /** Default agent name for `oh-my-opencode run` (env: OPENCODE_DEFAULT_AGENT) */
  default_run_agent: z.string().optional(),
  disabled_mcps: z.array(AnyMcpNameSchema).optional(),
  disabled_agents: z.array(z.string()).optional(),
  disabled_skills: z.array(BuiltinSkillNameSchema).optional(),
  disabled_hooks: z.array(z.string()).optional(),
  disabled_commands: z.array(BuiltinCommandNameSchema).optional(),
  /** Disable specific tools by name (e.g., ["todowrite", "todoread"]) */
  disabled_tools: z.array(z.string()).optional(),
  /** Enable hashline_edit tool/hook integrations (default: true at call site) */
  hashline_edit: z.boolean().optional(),
  /** Enable model fallback on API errors (default: false) */
  model_fallback: z.boolean().optional(),
  agents: AgentOverridesSchema.optional(),              // 14 个 Agent × 21 字段
  categories: CategoriesConfigSchema.optional(),         // 8 内置 + 自定义
  claude_code: ClaudeCodeConfigSchema.optional(),
  sisyphus_agent: SisyphusAgentConfigSchema.optional(),
  comment_checker: CommentCheckerConfigSchema.optional(),
  experimental: ExperimentalConfigSchema.optional(),
  auto_update: z.boolean().optional(),
  skills: SkillsConfigSchema.optional(),
  ralph_loop: RalphLoopConfigSchema.optional(),
  runtime_fallback: z.union([z.boolean(), RuntimeFallbackConfigSchema]).optional(),
  background_task: BackgroundTaskConfigSchema.optional(),
  notification: NotificationConfigSchema.optional(),
  babysitting: BabysittingConfigSchema.optional(),
  git_master: GitMasterConfigSchema.optional(),
  browser_automation_engine: BrowserAutomationConfigSchema.optional(),
  websearch: WebsearchConfigSchema.optional(),
  tmux: TmuxConfigSchema.optional(),
  sisyphus: SisyphusConfigSchema.optional(),
  /** Migration history to prevent re-applying migrations */
  _migrations: z.array(z.string()).optional(),
})

export type OhMyOpenCodeConfig = z.infer<typeof OhMyOpenCodeConfigSchema>
```

子 Schema 文件 (22+)

```
src/config/schema/
├── oh-my-opencode-config.ts      # 根 Schema
├── agent-overrides.ts            # Agent 覆盖 (14 个 Agent)
├── categories.ts                 # 分类系统
├── claude-code.ts                # Claude Code 兼容配置
├── experimental.ts               # 实验性功能
├── skills.ts                     # 技能配置
├── background-task.ts            # 后台任务配置
├── notification.ts               # 通知配置
├── babysitting.ts                # 不稳定 Agent 保姆配置
├── browser-automation.ts         # 浏览器自动化
├── comment-checker.ts            # 注释检查器
├── git-master.ts                 # Git 操作配置
├── ralph-loop.ts                 # Ralph 循环配置
├── runtime-fallback.ts           # 运行时 fallback
├── sisyphus-agent.ts             # Sisyphus Agent 配置
├── sisyphus.ts                   # Sisyphus 系统配置
├── tmux.ts                       # Tmux 配置
├── websearch.ts                  # 网络搜索配置
└── ...
```

Agent 覆盖配置

每个 Agent 支持 21 个可覆盖字段：

```
type AgentOverrideConfig = Partial<AgentConfig> & {
  prompt_append?: string          // 追加到 Agent prompt
  variant?: string                // 模型变体 (如 "high", "max")
  fallback_models?: string | string[]  // 自定义 fallback 链
}
```

示例配置：

```
{
  "agents": {
    "oracle": {
      "model": "anthropic/claude-opus-4-6",
      "variant": "max",
      "temperature": 0.05
    },
    "explore": {
      "model": "openai/gpt-5-nano",
      "prompt_append": "Always include file line numbers."
    }
  }
}
```

Config Handler — 6 阶段加载管线

> 源文件：`src/plugin-handlers/config-handler.ts` (49 行，完整源码)

```ts
// src/plugin-handlers/config-handler.ts — 完整源码
export interface ConfigHandlerDeps {
  ctx: { directory: string; client?: any }
  pluginConfig: OhMyOpenCodeConfig
  modelCacheState: ModelCacheState
}

export function createConfigHandler(deps: ConfigHandlerDeps) {
  const { ctx, pluginConfig, modelCacheState } = deps

  return async (config: Record<string, unknown>) => {
    const formatterConfig = config.formatter            // 保存 formatter（防止被覆盖）

    // Phase 1: Provider 配置 — 缓存模型 context limits
    applyProviderConfig({ config, modelCacheState })

    // Phase 2: 加载插件组件（10s 超时）
    const pluginComponents = await loadPluginComponents({ pluginConfig })

    // Phase 3: Agent 配置 — 合并内置/Claude/用户 Agent + 技能发现 + 模型解析
    const agentResult = await applyAgentConfig({
      config, pluginConfig, ctx, pluginComponents,
    })

    // Phase 4: 工具配置
    applyToolConfig({ config, pluginConfig, agentResult })

    // Phase 5: MCP 配置
    await applyMcpConfig({ config, pluginConfig, pluginComponents })

    // Phase 6: 命令配置
    await applyCommandConfig({ config, pluginConfig, ctx, pluginComponents })

    config.formatter = formatterConfig                  // 恢复 formatter
  }
}
```

Feature 模块 (19 个)

`src/features/` 下每个目录是一个独立功能模块：

核心功能

模块	源目录	功能
background-agent	`features/background-agent/`	后台 Agent 调度 + 并发控制
boulder-state	`features/boulder-state/`	Sisyphus 活跃计划状态追踪
context-injector	`features/context-injector/`	上下文注入（消息变换 hook）
hook-message-injector	`features/hook-message-injector/`	Hook 消息注入机制

背景代理子系统

`features/background-agent/` — 最复杂的 Feature 模块

```
background-agent/
├── manager.ts          # BackgroundManager（核心状态管理, ~1600 行）
├── concurrency.ts      # ConcurrencyManager（并发控制）
├── launcher.ts         # 任务启动器
├── poller.ts           # 任务轮询器
├── types.ts            # 类型定义
└── index.ts            # 导出
```

BackgroundManager 核心结构（源文件 1643 行）：

```
// src/features/background-agent/manager.ts — 核心类型和导入
import type { PluginInput } from "@opencode-ai/plugin"
import type { BackgroundTask, LaunchInput, ResumeInput } from "./types"
import { TaskHistory } from "./task-history"
import { ConcurrencyManager } from "./concurrency"
import { shouldRetryError, hasMoreFallbacks } from "../../shared/model-error-classifier
import { tryFallbackRetry } from "./fallback-retry-handler"
import { registerManagerForCleanup, unregisterManagerForCleanup } from "./process-clean
import { pruneStaleTasksAndNotifications, checkAndInterruptStaleTasks } from "./task-po

type OpencodeClient = PluginInput["client"]

// 内部类型
interface QueueItem {
  task: BackgroundTask
  input: LaunchInput
  resolve: (value: BackgroundTask) => void
  reject: (reason?: unknown) => void
}
```

ConcurrencyManager：

```
class ConcurrencyManager {
  // 并发限制优先级:
  // modelConcurrency > providerConcurrency > defaultConcurrency > 5
  getConcurrencyLimit(model): number

  // 信号量机制
  async acquire(model): Promise<void>  // 超限则排队
  release(model): void                 // 释放 → 移交
}
```

技能系统

模块	源目录	功能
opencode-skill-loader	`features/opencode-skill-loader/`	技能发现/加载/合并
skill-mcp-manager	`features/skill-mcp-manager/`	技能内嵌 MCP 生命周期
builtin-skills	`features/builtin-skills/`	内置技能实现

技能发现流程

> 源文件: `src/features/opencode-skill-loader/loader.ts` (148 行) — `discoverAllSkills()`

```
// src/features/opencode-skill-loader/loader.ts — discoverAllSkills 实现
export async function discoverAllSkills(directory?: string): Promise<LoadedSkill[]> {
  // 6 个路径并行加载
  const [opencodeProjectSkills, opencodeGlobalSkills, projectSkills,
         userSkills, agentsProjectSkills, agentsGlobalSkills] =
    await Promise.all([
      discoverOpencodeProjectSkills(directory),   // .opencode/skills/
      discoverOpencodeGlobalSkills(),              // ~/.config/opencode/skills/
      discoverProjectClaudeSkills(directory),      // .claude/skills/
      discoverUserClaudeSkills(),                  // ~/.claude/skills/
      discoverProjectAgentsSkills(directory),      // .agents/skills/
      discoverGlobalAgentsSkills(),                // ~/.agents/skills/
    ])

  // Priority: opencode-project > opencode > project (.claude + .agents) > user (.claud
  return deduplicateSkillsByName([
    ...opencodeProjectSkills,
    ...opencodeGlobalSkills,
    ...projectSkills,
    ...agentsProjectSkills,
    ...userSkills,
    ...agentsGlobalSkills,
  ])
}
```

各路径发现函数 — 统一模式:

```
// 每个路径的发现函数都是相同模式:
export async function discoverOpencodeProjectSkills(directory?: string): Promise<Loaded
  const opencodeProjectDir = join(directory ?? process.cwd(), ".opencode", "skills")
  return loadSkillsFromDir({ skillsDir: opencodeProjectDir, scope: "opencode-project" }
}
// scope 值: "builtin" | "config" | "user" | "project" | "opencode" | "opencode-project"
```

Claude Code 兼容

模块	源目录	功能
claude-code-agent-loader	`features/claude-code-agent-loader/`	从 `.claude/agents/` 加载 Agent
claude-code-command-loader	`features/claude-code-command-loader/`	从 `.claude/commands/` 加载命令
claude-code-mcp-loader	`features/claude-code-mcp-loader/`	从 `.mcp.json` 加载 MCP
claude-code-plugin-loader	`features/claude-code-plugin-loader/`	加载 Claude Code 插件
claude-code-session-state	`features/claude-code-session-state/`	会话状态兼容

MCP JSON 加载

支持 `${VAR}` 环境变量展开：

```
// .mcp.json
{
  "mcpServers": {
    "my-server": {
      "command": "node",
      "args": ["${HOME}/mcp/server.js"],
      "env": {
        "API_KEY": "${MY_API_KEY}"
      }
    }
  }
}
```

任务系统

模块	源目录	功能
claude-tasks	`features/claude-tasks/`	新任务系统（替代 TodoWrite）
run-continuation-state	`features/run-continuation-state/`	运行继续状态

Tmux 支持

模块	源目录	功能
tmux-subagent	`features/tmux-subagent/`	Tmux 多面板子代理管理

```
class TmuxSessionManager {
  // Tmux 配置
  config = {
    enabled: boolean,
    layout: "main-vertical" | ...,
    main_pane_size: number,      // 60%
    main_pane_min_width: number, // 120
    agent_pane_min_width: number // 40
  }

  onSessionCreated(event): void  // 子代理 session 创建时新建 pane
  cleanup(): void                // 清理所有 pane
}
```

其他功能

模块	源目录	功能
task-toast-manager	`features/task-toast-manager/`	task() 执行 Toast 通知
tool-metadata-store	`features/tool-metadata-store/`	工具执行元数据持久化
mcp-oauth	`features/mcp-oauth/`	MCP OAuth 认证流程
builtin-commands	`features/builtin-commands/`	内置命令（/start-work）

三层 MCP 体系

架构概览

```
Layer 1: 内置 MCP（3 个远程 HTTP）
  ├── websearch（Exa 或 Tavily）
  ├── context7
  └── grep_app

Layer 2: Claude Code MCP（从 .mcp.json）
  └── 支持 ${VAR} 环境变量展开
  └── stdio + HTTP 传输

Layer 3: 技能内嵌 MCP（从 SKILL.md YAML）
  └── 由 SkillMcpManager 管理
  └── stdio + HTTP 传输
```

内置 MCP

> 源文件: `src/mcp/index.ts` (35 行，完整源码)

```ts
// src/mcp/index.ts — 完整源码
type RemoteMcpConfig = {
  type: "remote"
  url: string
  enabled: boolean
  headers?: Record<string, string>
  oauth?: false
}

export function createBuiltinMcps(disabledMcps: string[] = [], config?: OhMyOpenCodeCon
  const mcps: Record<string, RemoteMcpConfig> = {}

  if (!disabledMcps.includes("websearch")) {
    mcps.websearch = createWebsearchConfig(config?.websearch)  // Exa 或 Tavily
  }

  if (!disabledMcps.includes("context7")) {
    mcps.context7 = context7                                   // mcp.context7.com/mcp
  }

  if (!disabledMcps.includes("grep_app")) {
    mcps.grep_app = grep_app                                   // mcp.grep.app
  }

  return mcps
}
```

MCP	URL	认证	用途
websearch	Exa: `mcp.exa.ai/mcp` / Tavily: `mcp.tavily.com/mcp/`	EXA_API_KEY 或 TAVILY_API_KEY	网络搜索
context7	`mcp.context7.com/mcp`	CONTEXT7_API_KEY (可选)	文档上下文
grep_app	`mcp.grep.app`	无	代码搜索

配置覆盖：

```
{
  "websearch": {
    "provider": "tavily"  // 默认 "exa"
  },
  "disabled_mcps": ["grep_app"]  // 禁用特定 MCP
}
```

Shared 工具库

`src/shared/` — 100+ 工具函数，13 个分类：

分类	代表文件	功能
日志	`logger.ts`	写入 /tmp/oh-my-opencode.log
模型	`model-requirements.ts`	Agent/Category fallback 链
合并	`merge-categories.ts`	分类配置合并
缓存	`provider-cache.ts`	Provider/模型缓存
权限	`permission-compat.ts`	Agent 工具权限
截断	`truncate-description.ts`	描述截断
规范化	`normalize-sdk-response.ts`	SDK 响应规范化
认证	`server-auth.ts`	服务端认证注入
名称	`agent-display-names.ts`	Agent 显示名映射
配置错误	`config-errors.ts`	配置加载错误处理
首次消息	`first-message-variant.ts`	首次消息变体门控
模型解析	`model-resolution.ts`	3 步模型解析管线
Env 解析	`env-resolver.ts`	环境变量解析

模型需求系统

`src/shared/model-requirements.ts`：

```
type FallbackEntry = {
  providers: string[]     // ["anthropic", "github-copilot"]
  model: string           // "claude-opus-4-6"
  variant?: string        // "max"
}

type ModelRequirement = {
  fallbackChain: FallbackEntry[]
  variant?: string
  requiresModel?: string
  requiresAnyModel?: boolean
  requiresProvider?: string[]
}

// 每个 Agent 和 Category 都定义了自己的 ModelRequirement
const AGENT_MODEL_REQUIREMENTS: Record<string, ModelRequirement> = { ... }
const CATEGORY_MODEL_REQUIREMENTS: Record<string, ModelRequirement> = { ... }
```

CLI 系统

`src/cli/` — Commander.js 子命令：

命令	文件	功能
`install`	`cli/install.ts`	交互式安装向导
`run`	`cli/run.ts`	非交互会话
`doctor`	`cli/doctor/`	健康诊断（多项检查）
`mcp-oauth`	`cli/mcp-oauth.ts`	MCP OAuth 认证设置

Doctor 检查

```
src/cli/doctor/checks/
├── node-version.ts     # Node.js 版本检查
├── bun-version.ts      # Bun 版本检查
├── opencode-version.ts # OpenCode 版本检查
├── config-valid.ts     # 配置文件验证
├── mcp-connection.ts   # MCP 连接测试
├── provider-auth.ts    # Provider 认证检查
└── index.ts            # 注册所有检查
```

构建与测试

构建

```
bun run build
# → bun build (ESM) + tsc --emitDeclarationOnly
# → externals: @ast-grep/napi
# → 输出: dist/
```

测试

```
bun test
# → Bun test suite
# → co-located *.test.ts
# → given/when/then 风格 (nested describe)
# → test-setup.ts 预加载 (bunfig.toml)
```

测试约定:

```
describe("ModuleName", () => {
  describe("#given specific condition", () => {
    describe("#when action happens", () => {
      it("#then expected result", () => {
        // ...
      })
    })
  })
})
```

CI/CD

工作流	触发	功能
`ci.yml`	push/PR	测试（分割：mock-heavy 隔离 + 批量）、typecheck、build、schema 自动提交
`publish.yml`	手动	版本号、npm publish、平台二进制、GitHub release、merge to master
`publish-platform.yml`	被调用	11 个平台二进制 via bun compile
`sisyphus-agent.yml`	@mention	AI Agent 处理 issues/PRs